

Modelado y Programación, Proyecto 1: Chat

Mayté García Franco

Septiembre 2026

1 Resumen

En este primer proyecto de Modelado y Programación se pidió crear un chat tanto del lado del servidor como del lado del cliente. Este proyecto toma como base el protocolo enviado al inicio del semestre, buscando que tanto el servidor como el cliente no solo sirvan en mi ambiente, sino que también operen con naturalidad con los proyectos del resto de mis compañeros.

2 Introducción

En este proyecto se tocaron conceptos que no había tocado con anterioridad en la carrera, tales como conectividad y el modelo cliente-servidor. También se aplicó por primera vez, con una mucha mayor consciencia, lo que es la programación concurrente y sistemas de construcción. Otro elemento importante sobre este proyecto es que me forzó a usar lenguajes de programación que nunca había imaginado que usaría, resultando no solo en un conocimiento teórico más amplio, sino en un pequeño pero importante inicio en la ampliación de mis habilidades técnicas.

3 Análisis y Diseño

3.1 Investigación de Cliente-Servidor

El servidor espera la petición del cliente por un puerto específico. Cuando el cliente realice la petición, el servidor abrirá una conexión. Esta petición se hace por un puerto previamente establecido y, en caso de ser aceptada, se crea una conexión donde el servidor y el cliente pueden intercambiar información.

3.1.1 Puertos

Existen 65535 puertos disponibles para cada dirección IP. Sin embargo, hay algunos que están prohibidos usar ya que están designados a servicios del sistema. Estos serían los puertos del 0 al 1023 ya que están reservados para procesos del

sistema. Los puertos del 1024 al 49151 pueden usarse libremente al igual que los puertos del 49152 al 65535, solo que estos últimos son de tipo temporal.

3.1.2 Sockets

Un socket nos va a permitir la comunicación bidireccional entre procesos a través de la red. Veamos como funcionan los sockets de acuerdo al modelo cliente-servidor:

Servidor:

- Creación del socket: el servidor crea un socket y lo asocia a una dirección IP y un número de puerto específico.
- Escucha: el socket del servidor espera conexiones entrantes de clientes.
- Aceptación de conexiones: cuando un cliente intenta conectarse, el servidor puede aceptar la conexión.

Cliente:

- Creación del socket: el cliente crea un socket sin asociarlo a una dirección específica.
- Conexión al servidor: el socket del cliente intenta conectarse al socket del servidor utilizando la dirección IP y el puerto del servidor.
- Comunicación: ya establecida la conexión, hay una comunicación bilateral.

3.2 División del problema

Ya con un conocimiento básico de cómo debería funcionar el Cliente y el Servidor junto con el protocolo, lo siguiente es realizar el famoso “divide y vencerás”.

Para el servidor tendríamos lo siguiente:

- Usuarios
 - Identifica
 - Cambia status
 - Desconecta
- Mensajes
 - Recibe mensaje
 - Envía mensaje
- Salas
 - Crear sala
 - Invitar usuario

- Agregar usuario
- Eliminar usuario
- Lista de usuarios

Para el cliente tendríamos lo siguiente:

- Mensajes
 - Recibe mensaje
 - Manda mensaje
- Salas
 - Crear sala
 - Invitar usuario
 - Aceptar invitación
 - Abandonar sala
 - Ver lista de usuarios
- Cambiar status
- Abandonar servidor

3.3 Diseño de clases

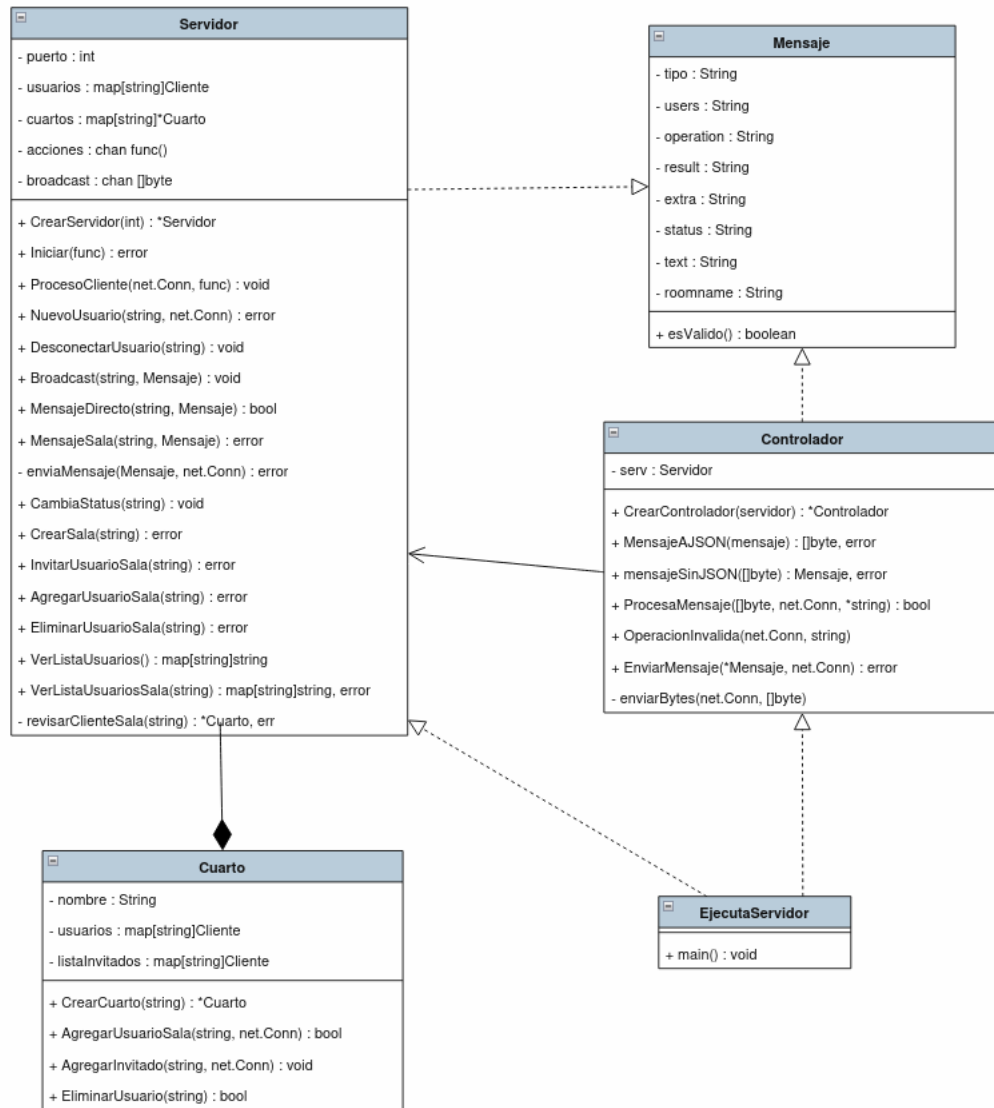


Figure 1: Diagrama de clases del Servidor.

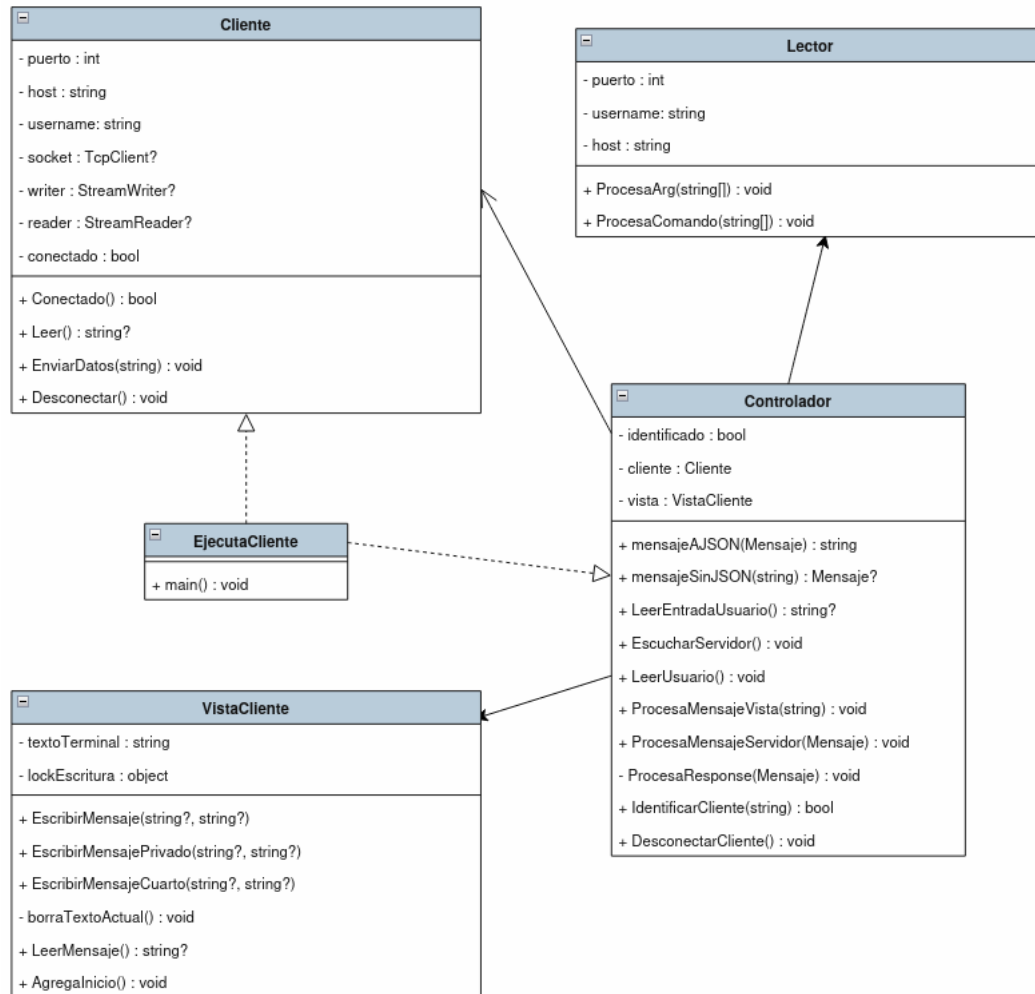


Figure 2: Diagrama de clases del Cliente.

4 Implementación

Para este proyecto se decidió programar el servidor en **Go** debido a que es un lenguaje con una implementación “sencilla” de concurrencia. De igual forma, su recolector de basura ayudará a facilitar el manejo del servidor y los recursos que pueda utilizar.

Del lado del cliente se decidió que se programará en **C#** ya que tiene un recolector de basura que trabaja de forma más “entendible” o, desde mi punto de vista, interesante a comparación de C o C++. Otro punto a favor de C# es que se caracteriza por ser orientado a objetos, por lo que me facilitará un poco

más la implementación de este.

En cuanto al sistema de construcción, se usará Taskfile.

5 Conclusiones

Fue un proyecto que me resultó desafiante pero bastante entretenido. Manejar los conceptos relacionados a la conectividad fue de lo más complicado para mí en este proyecto ya que me costaba entender como funciona cada uno de los elementos para lograr la comunicación, pero ya terminado el proyecto me siento capaz de decir que ya entiendo aunque sea un poco más como funciona todo esto.

Tenía la idea de que antes de programar cualquier programa considerado “difícil” para mí necesitaría un conocimiento profundo del lenguaje, pero con este proyecto caigo en cuenta que no es necesario saberlo todo desde un inicio, mientras uno trabaja más y más con el lenguaje se va acostumbrando automáticamente a él y quizás no pueda ser considerado un experto, pero uno se vuelve capaz de hacer lo necesario para completar lo que necesita. También se me complicó un poco el cambio de paradigma de programación, específicamente con Golang ya que este lenguaje no es específicamente orientado a objetos, por lo que me vi obligada a aunque sea tratar de pensar de forma distinta. Aún así, esto fue divertido.

Algo en lo que más me hizo autorreflexionar este proyecto fue la gran procrastinación que tengo y que debo arreglarla sí o sí. Este proyecto lo inicié aproximadamente 2 o 3 semanas después de que se mencionó por primera vez. Con ese retraso de semanas me di cuenta que si lo hubiera iniciado apenas pude, seguro hubiera logrado un resultado muchísimo mejor, haciendo que en este momento me sienta arrepentida, pero realmente agradezco que este proyecto me haya abierto los ojos y me haga buscar soluciones para resolver este grave problema de procrastinación.

Ya con este proyecto concluido, tengo como objetivo que para el siguiente proyecto inicie mi análisis y diseño apenas tenga oportunidad, buscando un resultado considerablemente bueno y sólido. También quiero tratar de pensar más como programadora para poder dividir todavía mejor el problema. Siento que justo un problema que tuve en este proyecto es que no conocía en su totalidad lo que implicaba realizar un servidor y un cliente, por lo que no pude dividir de la mejor forma el problema y, en consecuencia, mi diseño inicial es bastante diferente a como resultó.

Para finalizar, quiero destacar que este proyecto no solo me ayudo a poner en práctica mejores formas de programar y a desarrollar más habilidades técnicas, también fue un periodo de autorreflexión en cuanto a mis responsabilidades y cómo quiero mejorar mi forma de crear programas.

6 Bibliografía

- Universidad Nacional Autónoma de México. (UNAM). (s.f.) *UAPA - Sockets*. CUAED - UNAM. Recuperado de: <https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/Sockets/index.html>
- OnMex. (s.f.). *Qué es un socket*. Tecnología y Desarrollo. Recuperado de: <https://onmex.mx/tecnologia-y-desarrollo/que-es-un-socket/>
- Taskfile. (s.f.). *Guide/Documentation*. Taskfile.dev. Recuperado de: <https://taskfile.dev/docs/guide>
- Apuntes.de. (s.f.). Apuntes de Golang. Apuntes.de. Recuperado de: <https://apuntes.de/golang/gsc.tab=0>
- Microsoft Learn. (s.f.). *Recorrido por C#*. Microsoft Documentation. Recuperado de: <https://learn.microsoft.com/es-es/dotnet/csharp/tour-of-csharp/tutorials/>